
JEV-Star: Fast, Low-Cost StarCraft II Control with Language-Model Planning

Weiyu Ma Liangbing Zhao Yongcheng Zeng¹ Jian Zhao²

¹Institute of Automation, Chinese Academy of Sciences

²Beijing Zhongguancun Academy

sc2meisah@gmail.com jianzhao@zgci.ac.cn

Abstract

We present JEV-Star, a StarCraft II controller that defeats the strongest non-cheating built-in AI, Lv7, by combining fast JEV action selection with persistent GPT-6 planning. The combined system wins four full games at Lv5–Lv7, including two Lv7 victories with different seeds, while retaining a median JEV response time of 0.422 seconds. Mean estimated model cost across these games is USD 3.71 per game: USD 0.15 for JEV and USD 3.56 for GPT-6. We compare this system with an initial JEV-only controller in full-game macro control and multi-unit micromanagement. The standalone controller reaches a 20-minute limit against Lv2 without expanding. Across 35 battle maps with three episodes per map, the combined system raises mean enemy elimination from 16.50% to 37.69% and wins from 3 to 7 out of 105. Replay frames and decision logs connect these gains to resource reservation, persistent economic goals, and stable army objectives. The results demonstrate a practical division between inexpensive, subsecond decisions and longer-horizon planning. The comparison evaluates complete systems; concurrent interface improvements mean that planning’s contribution is not isolated by a controlled ablation.

1 Introduction

StarCraft II requires both immediate decisions and commitments whose benefits arrive much later. A controller can repeatedly choose legal, affordable actions yet never save enough to expand its economy. It can also alternate between attacking and retreating without carrying either decision through. These failures motivate a separation between choosing the next action and maintaining a strategy.

We implement this separation with JEV, a hosted model that selects among structured alternatives [2], and GPT-6, which supplies persistent strategic guidance. JEV remains responsible for frequent action selection. GPT-6 specifies goals, spending priorities, and conditions for changing army posture. The intended benefit is to preserve fast, low-cost interaction while using general-purpose planning where a single-step decision is insufficient.

JEV-Star builds on LLM Play SC2 for full-game control and SMAC-Hard for micromanagement [5, 11]. We compare JEV-only with JEV + GPT-6. The combined controller achieves two victories against Lv7, the highest non-cheating difficulty of StarCraft II’s built-in AI [3]. Our experiments measure game outcomes, response latency, and the combined model cost of a complete game. Replay frames and decision logs show how persistent plans change economic development and army behavior while preserving a fast action loop.

2 Related work

StarCraft II AI spans the SC2LE interface, full-game agents such as TStarBots, AlphaStar, TStarBot-X, and StarCraft Commander, and offline learning with AlphaStar Unplugged [7, 12, 18, 20, 21, 23]. Cooperative combat is studied in SMAC and its extensions, with methods including QMIX and MAPPO [5, 6, 15, 16, 26]. Our contribution concerns inference-time use of hosted models, without task-specific weight training.

LLM Play SC2 and LLM-PySC2 establish language-model interfaces for game control [9, 11]. ReAct, Reflexion, Voyager, and SayCan connect reasoning or persistent context to executable actions [1, 17, 22, 25]. AgentBench and WebArena emphasize evaluating interactive outcomes [10, 27]. JEV-Star uses a native choice interface for concrete actions and retains an explicit plan across decisions.

FrugalGPT and RouteLLM reduce cost through model selection, while ReWOO and LLMCompiler organize reasoning and tool execution [4, 8, 13, 24]. We assign models by control role: frequent JEV actions and occasional GPT-6 plans. Our experiments examine this assignment through game performance, response latency, and per-game expenditure.

3 Method

3.1 Structured action selection with JEV

At decision time t , the adapter constructs a structured observation x_t and an executable candidate set $\mathcal{A}_t$. JEV selects an identifier:

$$a_t = \text{JEV}(x_t, \mathcal{A}_t). \quad (1)$$

The executor validates the identifier, attempts the associated command, and includes recent outcomes in subsequent observations. Both methods use this loop; their difference is whether an explicit plan accompanies it (Figure 1).

For macro control, the observation summarizes resources, workers, buildings, technology, army composition, visible enemies, and recent actions. Candidates represent high-level commands such as producing a unit, constructing a building, researching an upgrade, expanding, attacking, retreating, or waiting. Local game code handles placement and low-level order execution. Feasibility checks consider prerequisites, supply, resources, and available producers. JEV chooses one command per request.

For micromanagement, the observation describes the team and visible enemies, including health, positions, ranges, cooldowns, and energy. Each living unit receives its own legal alternatives: move to an enumerated destination, attack an available target, heal an ally where supported, or stop. Unit choices share the same team observation and are submitted in concurrent batches. Execution returns information about rejected actions. This is centralized shared-vision control; it does not implement independently observed, decentralized MARL policies.

The JEV-only method selects directly from current state and short execution history. It has no separate process that maintains a future resource budget or a persistent team objective. In particular, an affordability filter can hide an expansion until sufficient minerals are available, even though reaching that threshold requires declining currently affordable purchases.

3.2 Persistent planning with GPT-6

In the combined method, a planner reads a state summary and, when available, the previous plan. It returns structured guidance z_k , which remains active across multiple action decisions:

$$z_k = \text{GPT-6}(x_{t_k}, z_{k-1}), \quad a_t = \text{JEV}(x_t, z_k, \tilde{\mathcal{A}}_t), \quad t_k \leq t < t_{k+1}. \quad (2)$$

The planner proposes goals rather than individual engine commands. Its output is checked before becoming active. A failed or delayed planning request leaves the previous valid plan available.

A macro plan specifies an economic objective, worker and base targets, production priorities, resource reservations, and an army posture. Resource reservation connects a future purchase to present action availability: when saving for an expansion, the adapter can suppress lower-priority spending

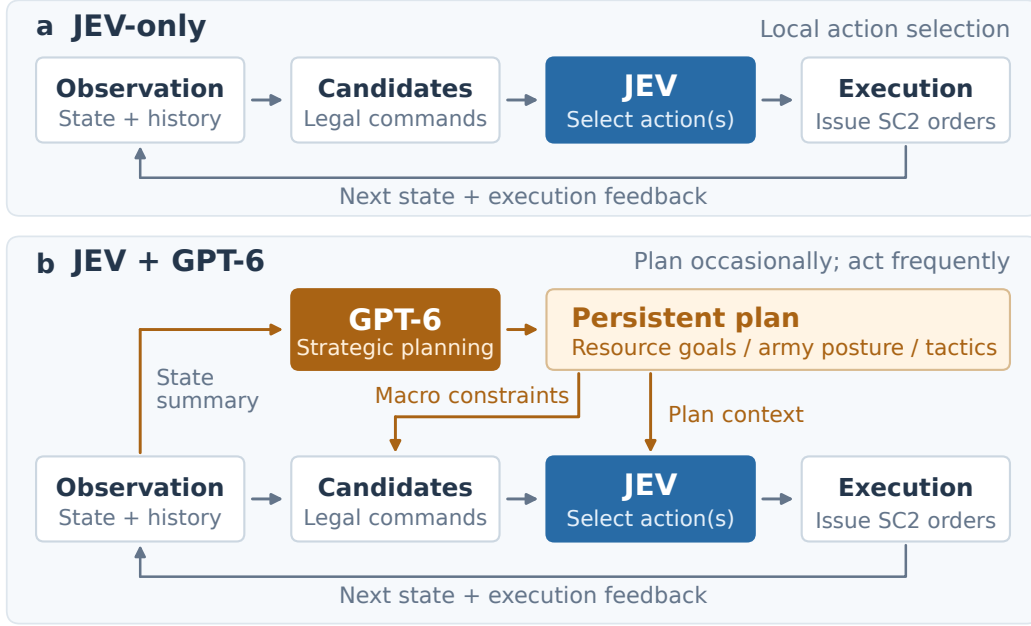


Figure 1: Two controllers with the same action loop. (a) JEV-only chooses executable actions from the current observation and recent feedback. (b) JEV + GPT-6 adds a plan that persists across action decisions. The plan supplies context to JEV and constrains macro candidates through resource reservations and army-posture rules. Micro plans provide tactical context while retaining legal unit choices. Macro and micro use separate game adapters.

until the required minerals accumulate. Emergency supply or defense actions can override these restrictions. Army guidance includes separate thresholds for starting and ending an attack and a minimum commitment interval, reducing repeated reversals near a single threshold. The plan enters both JEV’s context and the adapter’s candidate rules; the latter makes strategic commitments operational.

A micro plan describes conditional tactics and roles, such as which units should absorb pressure, which targets deserve priority, and when ranged units should reposition. One plan is generated for each map and reused for its three episodes. The legal action set remains available to JEV. A common plan provides shared intent, but independently chosen unit actions still lack a joint reservation mechanism for damage or healing targets.

3.3 Control timing

Full-game control is asynchronous: the game continues while requests are in flight. JEV requests are rate-limited to at most one per wall-clock second. The planner is checked on a nominal 60-game-second interval and at relevant events, subject to a minimum execution window. Completed plans are applied to subsequent decisions.

Micromanagement uses fixed-step interaction. The environment advances eight game loops only after a complete team decision has returned. Concurrent batching reduces waiting compared with sequential unit requests, but the slowest required response determines the next step. Consequently, model response latency and whole-episode wall time measure different parts of this system.

Neither method fine-tunes JEV or GPT-6. Both rely on the same division of responsibilities: game code determines observations and executable commands, JEV chooses actions, and the combined method adds GPT-6’s persistent goals.

Table 1: Completed full-game experiments (times in minutes:seconds). The first five rows form the main comparison. The final Lv8 row is a separate stress test against an opponent with an information advantage. A time limit is not a victory.

Method	Difficulty	Seed	Outcome	Game time	Wall time
JEV-only	Lv2	1	Time limit	20:00	20:40
JEV + GPT-6	Lv5	1	Win	13:28	14:00
JEV + GPT-6	Lv6	1	Win	10:49	11:18
JEV + GPT-6	Lv7	1	Win	12:48	13:26
JEV + GPT-6	Lv7	2	Win	15:55	16:31
JEV + GPT-6	Lv8	1	Loss	10:14	10:43

4 Experiments

4.1 Setup and measurements

We compare the initial JEV-only controller with the completed JEV + GPT-6 system. The runs use JEV 1.13, GPT-6 Astra with medium reasoning effort, and StarCraft II 5.0.16. Full-game experiments play Protoss against the built-in Zerg opponent on AltitudeLE, with eight starting workers. We report difficulty as Lv2, Lv5, Lv6, Lv7, or Lv8, following the ordered levels used by LLM Play SC2 [11]. The main sample contains one JEV-only game and four combined-system games; a completed Lv8 stress test is reported separately.

Micromanagement covers 35 local SMAC-Hard maps, with three episodes per map and method: 105 episodes each. Both methods use the same map configurations, game version, opponent controls, shared-vision setting, and disabled optional special abilities. Two local development maps are also excluded in a separate 33-map summary. There is one shared plan per map, so the three combined-method episodes do not represent independent planner samples. Candidate descriptions and execution handling also improved between the two controllers; these experiments measure the implemented systems rather than an isolated planner ablation.

We report wins, draws, losses, and mean enemy elimination, defined per episode as the fraction of initial enemy units eliminated. Each episode receives equal weight. For efficiency, we measure recorded response latency, total wall time, and model cost per complete episode. Macro outcomes and game durations are checked against replays. Cost estimates include recorded calls with known token usage, whether or not a returned plan was accepted.

4.2 Full-game performance: defeating the strongest non-cheating AI

Table 1 shows that JEV + GPT-6 wins all four completed ordinary-difficulty matches, including two victories against Lv7, the strongest built-in opponent without cheating advantages. These wins take 10:49–15:55 of game time. The first attacks occur between 7:22 and 8:11, following an economic and production buildup. JEV-only reaches its 20-minute game-time limit without a win. The two Lv7 seeds establish successful play at the highest non-cheating difficulty, although the sample is too small to estimate a reliable win rate.

Figure 2 shows four moments from the Lv7 seed-1 victory: expansion, army movement, pressure on a Zerg base, and the final push. The frames come from the evaluated replay, rendered afterward with an observer camera. This viewing perspective supplies illustrations only; the controller acts on structured observations, not these images.

LLM Play SC2 reports its main evaluation at Lv5 (12 wins in 20 games for GPT-4-Turbo); a separate GPT-3.5 prompt study reaches Lv6 with an 8.33% win rate [11]. Our Lv7 victories exceed the demonstrated difficulty in those reported experiments. The game versions and starting conditions differ, so this is a difficulty milestone rather than a matched win-rate comparison. That paper does not provide directly comparable per-game model charges or response latencies; we therefore quantify JEV-Star’s speed and expenditure from our own logs in Section 4.5.

Stress test with privileged vision. We additionally test Lv8, whose built-in opponent has a vision advantage. This game ends in defeat at 10:14. At five minutes, the agent’s army resource value is

a 03:12 | Natural expansion**b 07:43 | Army advances****c 08:25 | Attack on the Zerg base****d 12:45 | Final push before victory**

JEV + GPT-6 vs. Lv7 (seed 1) | AltitudeLE | Result: victory at 12:48

Figure 2: Actual replay frames from JEV + GPT-6 defeating Lv7 on AltitudeLE, seed 1. (a) The natural Nexus is under construction. (b) The army advances after the logged attack command at 07:33. (c) Units attack the Zerg base. (d) The push continues shortly before victory at 12:48. Displayed times are game time, rounded to the nearest second. The replay observer view is used only for presentation.

Table 2: Micromanagement outcomes. Elimination is the episode-average fraction of enemy units eliminated. The 33-map subset excludes two local development maps. Both methods use three episodes per map.

Method	All 35 maps: 105 episodes		33 maps: 99 episodes	
	W/D/L	Elimination	Wins	Elimination
JEV-only	3/2/100	16.50%	3/99	15.55%
JEV + GPT-6	7/0/98	37.69%	7/99	39.17%

900 versus the opponent’s 1,525; its first Immortal appears only at 6:25. It never issues an attack command. This failure illustrates a remaining economic–military imbalance when facing a stronger, privileged opponent; it is separate from the non-cheating difficulty milestone.

4.3 Micromanagement: better damage, limited conversion to wins

Table 2 summarizes the complete 35-map comparison. Mean enemy elimination rises from 16.50% to 37.69%, an increase of 21.19 percentage points, or 2.28 times the initial value. Wins increase from 3 to 7 out of 105. Excluding the two development maps retains the same win counts and increases mean elimination from 15.55% to 39.17%. The improvement is therefore not driven by those maps.

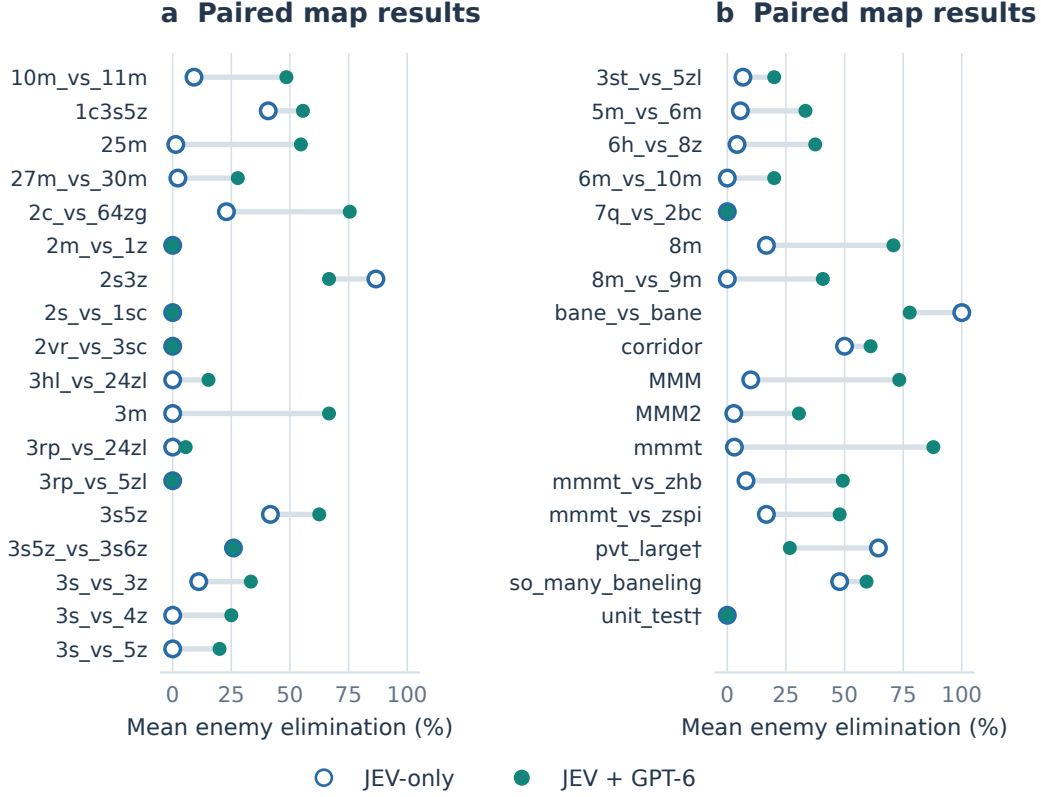


Figure 3: Mean enemy elimination on all 35 maps, with three episodes per point. A connecting line pairs the two methods on the same map. The combined system improves 25 maps, ties seven, and declines on three. Daggers mark the two local development maps excluded from the 33-map summary.

The combined system improves mean elimination on 25 maps, ties on seven, and declines on three (Figure 3). On `mmmt`, elimination increases from 3.03% to 87.88% and wins from zero to two. On `MMM`, it rises from 10.00% to 73.33%, with one win; `8m` and `2c_vs_64zg` also acquire wins. These cases indicate a useful change in tactical behavior beyond merely issuing valid actions.

Nevertheless, the combined system still loses 98 episodes. On `3m`, elimination increases from zero to 66.67% without a win. On `2s3z`, wins decline from two to one; on `bane_vs_bane`, elimination declines from 100% to 77.78%. Planning helps many battles but does not consistently convert local damage into team survival and victory.

Appendix A presents paired frames from the first `mmmt` episode of each method, together with the combined controller’s tactical guidance and recorded outcome. These examples illustrate the quantitative results; they are not additional evaluation episodes.

4.4 Why does planning help?

Saving for a future goal. The initial JEV-only full game illustrates a failure of longer-horizon resource allocation. It grows to 68 workers but remains on one base for the entire 20 minutes. Across 1,024 requests, the largest observed mineral balance is 335, below the 400 required for a Nexus. Expansion is consequently absent from every candidate set. Repeated affordable purchases prevent the saving needed to make the strategic action available.

The first accepted plan in a combined-system game requests two bases and instructs the controller to stop worker growth near 19 workers while saving for the natural expansion. The adapter turns this instruction into a pending reservation. Across the four wins, a second Nexus is first observed

Table 3: Plan–decision–execution trace from the Lv7 seed-1 victory. Rows summarize contemporaneous logs; time is minutes:seconds.milliseconds. A submitted building order is followed by an independent observation of construction.

Game time	Source	Recorded behavior
01:34.286	GPT-6 plan	Start the natural around 19 workers before filling army ceilings.
02:56.250	JEV observation	400 minerals; one Nexus; expansion is an available choice.
02:56.786	JEV and executor	Select BUILD NEXUS in 391 ms; submit one build order.
03:05.179	Game state	A second Nexus is observed, including construction.
07:32.321	GPT-6 plan	Attack posture; 44 ready army supply; 30-second commitment.
07:33.214	Executor	Submit 24 attack orders under the new plan.
12:47.679	Game result	Victory.

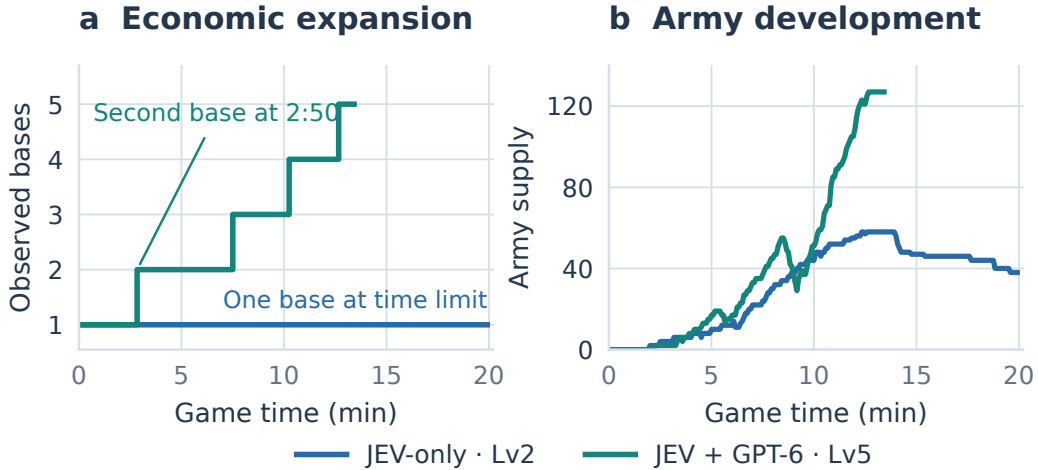


Figure 4: Example full-game trajectories: JEV-only at Lv2 and JEV + GPT-6 at Lv5. The standalone controller remains on one base; the combined controller saves for expansion and develops a larger army. Base counts include structures under construction, and army supply can include pending production. The different opponents make this a behavioral illustration, not a controlled ablation.

at 2:50–3:05, and peak observed base counts reach four to six. Figure 4 contrasts the economic and army trajectories. The useful contribution of planning is an explicit future target plus a mechanism that protects the resources needed to reach it.

This evidence diagnoses the initial controller’s planning limitation, rather than showing that JEV can never reason about expansion. The affordability filter and one-step objective interact: neither exposes the value of temporarily refusing purchases. The combined system supplies that missing context and commitment.

Table 3 follows the same Lv7 game shown in Figure 2. The log connects a request to prioritize the natural expansion, JEV’s choice when 400 minerals become available, and the subsequent appearance of a second Nexus. An attack plan later produces 24 unit orders within one game second. This trace distinguishes model intent, command submission, and observed game state. Selected original fields appear in Appendix A.

Maintaining an army objective. In the same JEV-only game, six attack commands and seven retreat commands include five attack-to-retreat reversals within two game seconds. Such rapid reversals leave little time for an attack to make progress. None of the four combined-system wins contains a reversal within that interval. Persistent posture, separate attack and retreat thresholds, and a commitment window stabilize the objective across JEV calls. Peak observed army supply rises from 58 in the standalone run to 79–129 in the combined games. Different opponents and other controller changes preclude a causal estimate, but the traces are consistent with better strategic continuity.

Table 4: Pooled latency over recorded responses with known usage. Macro JEV + GPT-6 statistics cover the four ordinary-difficulty wins; the Lv8 stress test is excluded. Micro statistics cover 105 episodes per method and 35 shared plans for the combined method.

Setting	Method	JEV action responses		GPT-6 planning responses	
		n	P50/P95 (s)	n	P50/P95 (s)
Macro	JEV-only	1,023	0.406/0.579	0	–
Macro	JEV + GPT-6	2,905	0.422/0.531	62	27.601/36.492
Micro	JEV-only	8,189	0.485/0.688	0	–
Micro	JEV + GPT-6	9,743	0.500/0.594	35	37.172/43.725

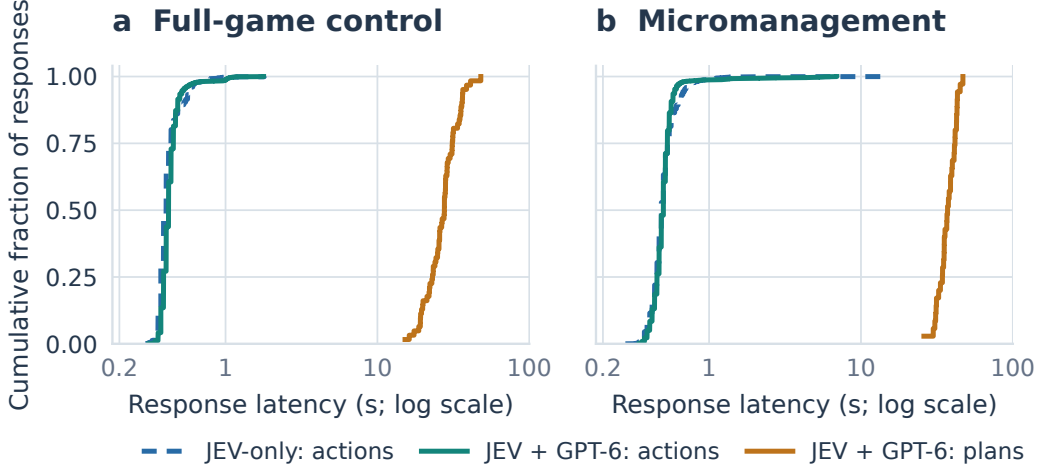


Figure 5: Response-latency distributions by role and method. Both methods retain subsecond median JEV responses. GPT-6 provides less frequent, more expensive planning calls. The horizontal axis is logarithmic; each curve pools responses within its setting.

Shared intent does not resolve all coordination. Micromanagement reveals a different boundary. In the combined system, 2,629 of 2,662 unit decisions with a ready weapon and an available in-range target choose an in-range attack. Basic engagement is usually sensible. However, among 930 decisions with a long weapon cooldown, nearby melee pressure, and an available movement endpoint that increases separation, 794 continue attacking and only 63 choose such a separating endpoint. The controller often fails to use movement during downtime.

Healing exposes resource conflicts between units. Execution records contain 500 healing rejections because a target is already targeted and ten for insufficient energy. Among 378 subsequent decisions that expose a previously rejected healing target together with an alternative legal target, 243 repeat the previous target. These conditional counts identify recurring behavior, not optimal-action labels. They suggest that planning alone cannot replace precise cooldown control, updated execution feedback, or joint allocation of healing targets.

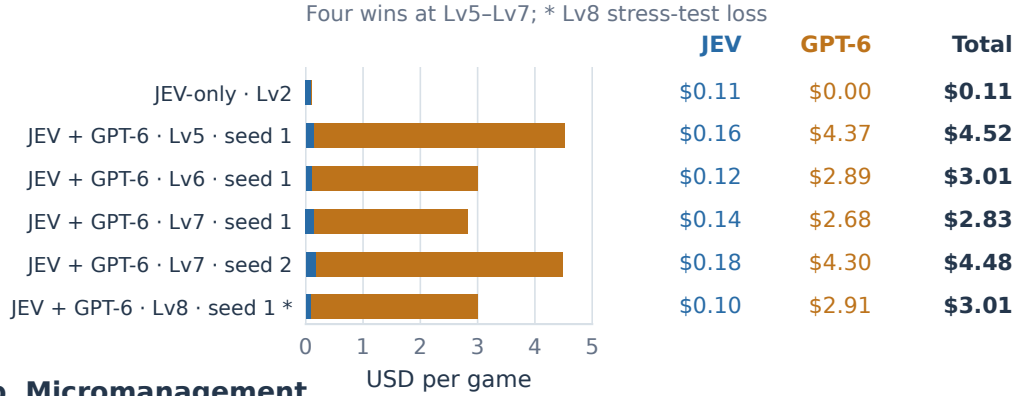
4.5 Response time and cost per game

Frequent actions remain subsecond. Table 4 and Figure 5 report response distributions. Median JEV latency is 0.406 seconds for standalone macro control and 0.422 seconds with planning; corresponding micro medians are 0.485 and 0.500 seconds. In the combined system, macro and micro JEV 95th percentiles are 0.531 and 0.594 seconds. GPT-6 planning is less frequent but slower: median latency is 27.60 seconds in macro control and 37.17 seconds in micromanagement. These are different computational roles and inputs, so their ratio is not a same-task model speedup. The practical benefit is that most actions proceed through the faster decision layer.

Table 5: Mean model cost and elapsed wall time per completed episode. Macro averages use the main full-game sample. Micro planner cost and setup time are amortized over three episodes per map. Costs are known-usage standard-API-equivalent estimates, not billing statements.

Setting	Method	Episodes	USD per episode			Wall time (s/episode)
			JEV	GPT-6	Total	
Macro	JEV-only	1	0.1069	0.0000	0.1069	1239.8
Macro	JEV + GPT-6	4	0.1499	3.5604	3.7103	828.6
Micro	JEV-only	105	0.0448	0.0000	0.0448	25.8
Micro	JEV + GPT-6	105	0.0588	0.0898	0.1486	46.2

a Full games



b Micromanagement

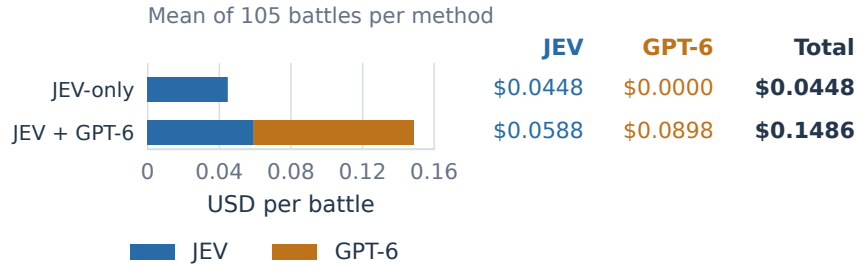


Figure 6: Cost per game, with model components and totals. Top: each completed full-game experiment, including the separate Lv8 defeat. Bottom: mean battle cost over 105 episodes per method, amortizing each map’s planning call across three episodes. Components and totals are rounded independently. Missing usage remains unpriced.

A complete game includes both models. Figure 6 shows the cost of each full game and the average micro episode, separating JEV, GPT-6, and their total. Across the four full-game wins, combined cost ranges from USD 2.83 to USD 4.52 and averages USD 3.7103. JEV accounts for USD 0.1499 per game and GPT-6 for USD 3.5604; planning contributes 96.0% of the total. JEV’s recorded macro usage costs approximately USD 0.21 per 1,000 responses. Cheap action selection therefore supports frequent interaction, while the planning schedule determines most of the game’s model expenditure.

For micromanagement, standalone JEV costs USD 0.0448 per episode. The combined method costs USD 0.1486, consisting of USD 0.0588 for JEV and USD 0.0898 for planning. Excluding the two development maps gives USD 0.0243 and USD 0.1168, respectively. The combined method thus spends more than JEV-only while producing better aggregate combat outcomes. Its advantage is improved capability with an inexpensive action layer, not a reduction in the total charge relative to the standalone controller.

Mean micro wall time increases from 25.78 to 46.23 seconds per episode. This includes amortized planning, environment setup, stepping, and cleanup. Macro games run in real time while inference overlaps game execution, and their durations also depend on when a match ends. Neither shorter successful games nor the difference between action and planning latency alone establishes a full-game speedup over prior systems.

Accounting basis. We price recorded token usage at the providers’ standard rates on September 22, 2026 [14, 19]. JEV charges for input; GPT-6 accounting separates cached from uncached input and includes output tokens. Reasoning output is a subset of output, not an additional charge. GPT-6 runs use Codex, so the reported amount is an equivalent API estimate. Calls without usage remain unpriced; Appendix C gives token totals, missing-call counts, and per-game figures.

5 Discussion and limitations

The experiments support a specific explanation for the standalone controller’s weakness: local action selection does not reliably create and preserve a future objective. Saving for an expansion requires declining immediately feasible actions; coordinated attacks require maintaining posture despite short-term fluctuations. GPT-6 supplies these commitments, and the adapter makes them affect action selection. This explains a substantial part of the observed behavior more directly than treating every game failure as an inability to choose legal commands.

The comparison remains observational. The standalone and combined controllers also differ in candidate descriptions and execution handling, and the macro opponents are not matched. Four successful macro games and three micro episodes per map cannot establish broad reliability. A controlled next experiment should use the same executor and seeds with the planner enabled or disabled, followed by separate tests of resource reservation and posture persistence. The current data motivate the planning hypothesis without isolating its entire contribution.

The scope is also restricted: one macro race and matchup, one macro map, a custom starting-worker count, centralized micro observations, and separate macro and micro implementations. Low battle win counts remain a material weakness. Finally, inexpensive JEV inference does not make the combined system’s planning free. Reducing unnecessary replanning or sharing plans more effectively is important because GPT-6 dominates observed full-game cost.

6 Conclusion

JEV-Star defeats Lv7, StarCraft II’s strongest non-cheating built-in AI, through a combination of rapid structured decisions and persistent planning. Compared with the initial JEV-only controller, it also raises mean enemy elimination from 16.50% to 37.69% across 35 battle maps. Resource reservation and stable army objectives help explain the improvement. The four full-game wins retain a 0.422-second median JEV response and cost an estimated USD 3.71 per game. JEV provides inexpensive action selection, while GPT-6 improves strategic organization and accounts for most model expenditure.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. URL <https://arxiv.org/abs/2204.01691>.
- [2] Diogo Almeida. Introducing system one models & jev. TypeSafe AI, 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Published September 15, 2026; accessed September 22, 2026.
- [3] Blizzard Entertainment. StarCraft II api protocol: built-in opponent difficulty levels, 2026. URL <https://github.com/Blizzard/s2client-proto/blob/master/s2clientprotocol/sc2api.proto>. Difficulty enumeration; accessed September 22, 2026.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*, 2023. URL <https://arxiv.org/abs/2305.05176>.

- [5] Yue Deng, Yan Yu, Weiyu Ma, Zirui Wang, Wenhui Zhu, Jian Zhao, and Yin Zhang. SMAC-Hard: Enabling mixed opponent strategy script and self-play on SMAC. *arXiv preprint arXiv:2412.17707*, 2024. URL <https://arxiv.org/abs/2412.17707>.
- [6] Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob N. Foerster, and Shimon Whiteson. SMACv2: An Improved Benchmark for Cooperative Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2212.07489*, 2022. URL <https://arxiv.org/abs/2212.07489>.
- [7] Lei Han, Jiechao Xiong, Peng Sun, Xinghai Sun, Meng Fang, Qingwei Guo, Qiaobo Chen, Tengfei Shi, Hongsheng Yu, Xipeng Wu, and Zhengyou Zhang. TStarBot-X: An Open-Sourced and Comprehensive Study for Efficient League Training in StarCraft II Full Game. *arXiv preprint arXiv:2011.13729*, 2020. URL <https://arxiv.org/abs/2011.13729>.
- [8] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM Compiler for Parallel Function Calling. *arXiv preprint arXiv:2312.04511*, 2023. URL <https://arxiv.org/abs/2312.04511>.
- [9] Zongyuan Li, Yanan Ni, Runnan Qi, Lumin Jiang, et al. LLM-PySC2: Starcraft II learning environment for Large Language Models. *arXiv preprint arXiv:2411.05348*, 2024. URL <https://arxiv.org/abs/2411.05348>.
- [10] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, et al. AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688*, 2023. URL <https://arxiv.org/abs/2308.03688>.
- [11] Weiyu Ma, Qirui Mi, Yongcheng Zeng, Xue Yan, Yuqiao Wu, Runji Lin, Haifeng Zhang, and Jun Wang. Large language models play StarCraft II: Benchmarks and a chain of summarization approach. *arXiv preprint arXiv:2312.11865*, 2023. URL <https://arxiv.org/abs/2312.11865>. Version 3, June 2024.
- [12] Michaël Mathieu, Sherjil Ozair, Srivatsan Srinivasan, Caglar Gulcehre, et al. AlphaStar Unplugged: Large-Scale Offline Reinforcement Learning. *arXiv preprint arXiv:2308.03526*, 2023. URL <https://arxiv.org/abs/2308.03526>.
- [13] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*, 2024. URL <https://arxiv.org/abs/2406.18665>.
- [14] OpenAI. gpt-6-astra model documentation and pricing, 2026. URL <https://developers.openai.com/api/docs/models/gpt-6-astra>. Standard API rates; accessed September 22, 2026.
- [15] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:1803.11485*, 2018. URL <https://arxiv.org/abs/1803.11485>.
- [16] Mikayel Samvelyan, Tabish Rashid, Christian Schröder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. *arXiv preprint arXiv:1902.04043*, 2019. URL <https://arxiv.org/abs/1902.04043>.
- [17] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [18] Peng Sun, Xinghai Sun, Lei Han, Jiechao Xiong, Qing Wang, Bo Li, Yang Zheng, Ji Liu, Yongsheng Liu, Han Liu, and Tong Zhang. TStarBots: Defeating the Cheating Level Builtin AI in StarCraft II in the Full Game. *arXiv preprint arXiv:1809.07193*, 2018. URL <https://arxiv.org/abs/1809.07193>.
- [19] TypeSafe AI. Models and pricing, 2026. URL <https://docs.typesafe.ai/models>. Accessed September 22, 2026.
- [20] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, et al. StarCraft II: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017. URL <https://arxiv.org/abs/1708.04782>.
- [21] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019. doi: 10.1038/s41586-019-1724-z. URL <https://www.nature.com/articles/s41586-019-1724-z>.
- [22] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [23] Xiangjun Wang, Junxiao Song, Penghui Qi, Peng Peng, Zhenkun Tang, Wei Zhang, Weimin Li, Xiongjun Pi, Jujie He, Chao Gao, Haitao Long, and Quan Yuan. SCC: an efficient deep reinforcement learning agent mastering the game of StarCraft II. *arXiv preprint arXiv:2012.13169*, 2020. URL <https://arxiv.org/abs/2012.13169>.
- [24] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. *arXiv preprint arXiv:2305.18323*, 2023. URL <https://arxiv.org/abs/2305.18323>.

- [25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [26] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. *arXiv preprint arXiv:2103.01955*, 2021. URL <https://arxiv.org/abs/2103.01955>.
- [27] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854*, 2023. URL <https://arxiv.org/abs/2307.13854>.

A Replay frames and decision traces

A.1 A paired micromanagement example

Figure 7 compares the first `mmmt` episode for each method. Both start with 22 allied and 22 enemy units. JEV-only loses at 12.14 seconds with two allied units remaining; JEV + GPT-6 wins at 21.43 seconds with nine. The winning controller’s recorded guidance calls for an infantry screen, supported tanks, and healing of wounded units. This is one illustrative pair: the complete map result is zero versus two wins out of three, reported alongside all other maps.

a JEV-only | 5.98 s



Surviving units: 19 allied / 22 enemy

b JEV + GPT-6 | 5.98 s



Surviving units: 21 allied / 21 enemy

c Loss | 12.14 s



Surviving units: 2 allied / 20 enemy

d Win | 21.43 s



Surviving units: 9 allied / 2 enemy

Figure 7: Replay frames from `mmmt`, episode 1. The top row shows both methods at the same game time, 5.98 seconds. The bottom row shows each episode’s terminal state, at different times. Allied units are blue and opponents orange. Counts include surviving non-attacking support units, so a benchmark victory can leave enemy Medivacs alive. The same camera settings and fixed image crop are used for both columns.

A.2 Full-game decision trace

This excerpt projects recorded fields from the Lv7 seed-1 log; times and response latency are rounded. Action identifier 21 denotes building a Nexus. The observation, JEV response, command, and game result are separate events.

```
[01:34.286] plan_accepted
  base_target: 2; worker_target: 30
  guidance (exact sentence):
    "Start the natural around 19 workers before filling army ceilings."

[02:56.250] request
  resource.mineral: 400; building.nexus_count: 1

[02:56.786] response
  answer.choice: "21" (BUILD NEXUS)
  answer.confidence: 0.83; latency_ms: 391

[02:56.786] action
  description: "BUILD NEXUS"
  orders_submitted: 1
  orders[0].ability: "PROTOSSBUILD_NEXUS"

[07:32.321] plan_accepted
  army_posture: "attack"
  attack_min_army: 44; retreat_below_army: 26
  min_posture_seconds: 30

[07:33.214] action
  description: "MULTI-ATTACK"
  orders_submitted: 24

[12:47.679] end
  result: "Victory"
```

A.3 Micromanagement plan and outcome

These sentences are quoted from the accepted tactical plan for mmmmt; the outcome fields come from its first episode. The plan is reused across the map's three episodes. Its requested behavior is distinct from whether each unit follows that behavior successfully.

```
plan_accepted | map: "mmmt"
opening (exact sentences):
  "Advance together east through the central corridor, y in [11,21), toward the raised center."
  "Keep infantry screening tanks and Medivacs behind infantry."
movement_and_cooldowns (exact sentences):
  "Ready, in-range shooters normally fire."
  "Withdraw a badly wounded unit taking recent damage toward healing while healthy allies maintain fire."
episode_end
  episode: 1; status: "win"
  final_game_loop: 480; game_seconds: 21.429
  final.allies_alive: 9; final.enemies_alive: 2
```

B Per-map micromanagement results

Each map has three episodes per method. The table reports wins and mean enemy elimination; daggers identify the two local development maps. All 35 maps are included in the main 105-episode result.

Table 6: Complete comparison of the two methods on the same maps.

Map	Wins / 3		Enemy elimination (%)	
	JEV-only	JEV + GPT-6	JEV-only	JEV + GPT-6
3m	0	0	0.0	66.7
8m	0	1	16.7	70.8
25m	0	0	1.3	54.7
5m_vs_6m	0	0	5.6	33.3
8m_vs_9m	0	0	0.0	40.7
10m_vs_11m	0	0	9.1	48.5
27m_vs_30m	0	0	2.2	27.8
MMM	0	1	10.0	73.3
MMM2	0	0	2.8	30.6
2s3z	2	1	86.7	66.7
3s5z	0	0	41.7	62.5
3s5z_vs_3s6z	0	0	25.9	25.9
3s_vs_3z	0	0	11.1	33.3
3s_vs_4z	0	0	0.0	25.0
3s_vs_5z	0	0	0.0	20.0
1c3s5z	0	0	40.7	55.6
2m_vs_1z	0	0	0.0	0.0
corridor	0	0	50.0	61.1
6h_vs_8z	0	0	4.2	37.5
2s_vs_1sc	0	0	0.0	0.0
so_many_baneling	0	0	47.9	59.4
bane_vs_bane	1	1	100.0	77.8
2c_vs_64zg	0	1	22.9	75.5
2vr_vs_3sc	0	0	0.0	0.0
3hl_vs_24zl	0	0	0.0	15.3
3rp_vs_5zl	0	0	0.0	0.0
3rp_vs_24zl	0	0	0.0	5.6
7q_vs_2bc	0	0	0.0	0.0
3st_vs_5zl	0	0	6.7	20.0
6m_vs_10m	0	0	0.0	20.0
mmmt_vs_zspi	0	0	16.7	47.9
unit_test [†]	0	0	0.0	0.0
mmmt	0	2	3.0	87.9
mmmt_vs_zhb	0	0	8.0	49.3
pvt_large [†]	0	0	64.4	26.7

C Cost accounting and reproducibility

Table 7 lists the charges underlying Figure 6. Each total is the sum of the two model components. The main combined macro average includes only the four Lv5–Lv7 games; the Lv8 stress test remains a separate completed result.

Table 7: Individual full-game costs estimated from known token usage.

Method	Difficulty	Seed	JEV (USD)	GPT-6 (USD)	Total (USD)
JEV-only	Lv2	1	0.1069	0.0000	0.1069
JEV + GPT-6	Lv5	1	0.1551	4.3683	4.5233
JEV + GPT-6	Lv6	1	0.1169	2.8885	3.0053
JEV + GPT-6	Lv7	1	0.1449	2.6842	2.8291
JEV + GPT-6	Lv7	2	0.1829	4.3006	4.4834
JEV + GPT-6	Lv8	1	0.1029	2.9068	3.0098

The rates are USD 0.042 per million JEV input tokens with no output charge, and USD 10, 1, and 50 per million GPT-6 uncached input, cached input, and output tokens, respectively [14, 19]. Cache-write input, if present, uses USD 12.50 per million. Requests exceeding 272,000 input tokens use the published long-context multipliers of two for input and 1.5 for output. We apply rates to each response before aggregation. These rates serve only to reconstruct game expenditure; the main comparison uses actual workload totals.

Table 8: Recorded token totals for the main samples. Cached input is included in GPT-6 input. Unpriced J/G counts identify JEV/GPT-6 attempts without recorded usage; their unknown cost is omitted from the known-usage estimate.

Setting	Method	JEV input	GPT-6 input	Cached input	GPT-6 output	Unpriced J/G
Macro	JEV-only	2,544,622	0	0	0	1/0
Macro	JEV + GPT-6	14,278,950	1,437,426	166,400	27,297	2/4
Micro	JEV-only	111,915,677	0	0	0	0/0
Micro	JEV + GPT-6	147,109,930	847,569	30,720	24,498	16/0

The Lv8 stress test additionally contains one GPT-6 attempt without usage. A canceled planning attempt before the selected micro run also lacks usage and falls outside the completed-episode totals. Neither is treated as free. Micro planning costs are divided by three because each map’s plan is reused for three episodes; they would be higher per game without reuse.

The accompanying analysis files contain 210 battle-episode records, response-level latency and usage, full-game action and state trajectories, and accepted plans. A selection manifest links these records to frozen source summaries and hashed event-log prefixes. Later appends to logs do not alter the selected data. Request latency is measured over responses with known usage; wall time comes from complete run summaries and includes setup and cleanup.

Reproduction requires the stated StarCraft II build, map files, shared-vision and ability settings, and the recorded action adapters. The initial macro catalog has 72 commands and the combined controller has 73; candidate and execution improvements are part of the system comparison. No model weights are trained.